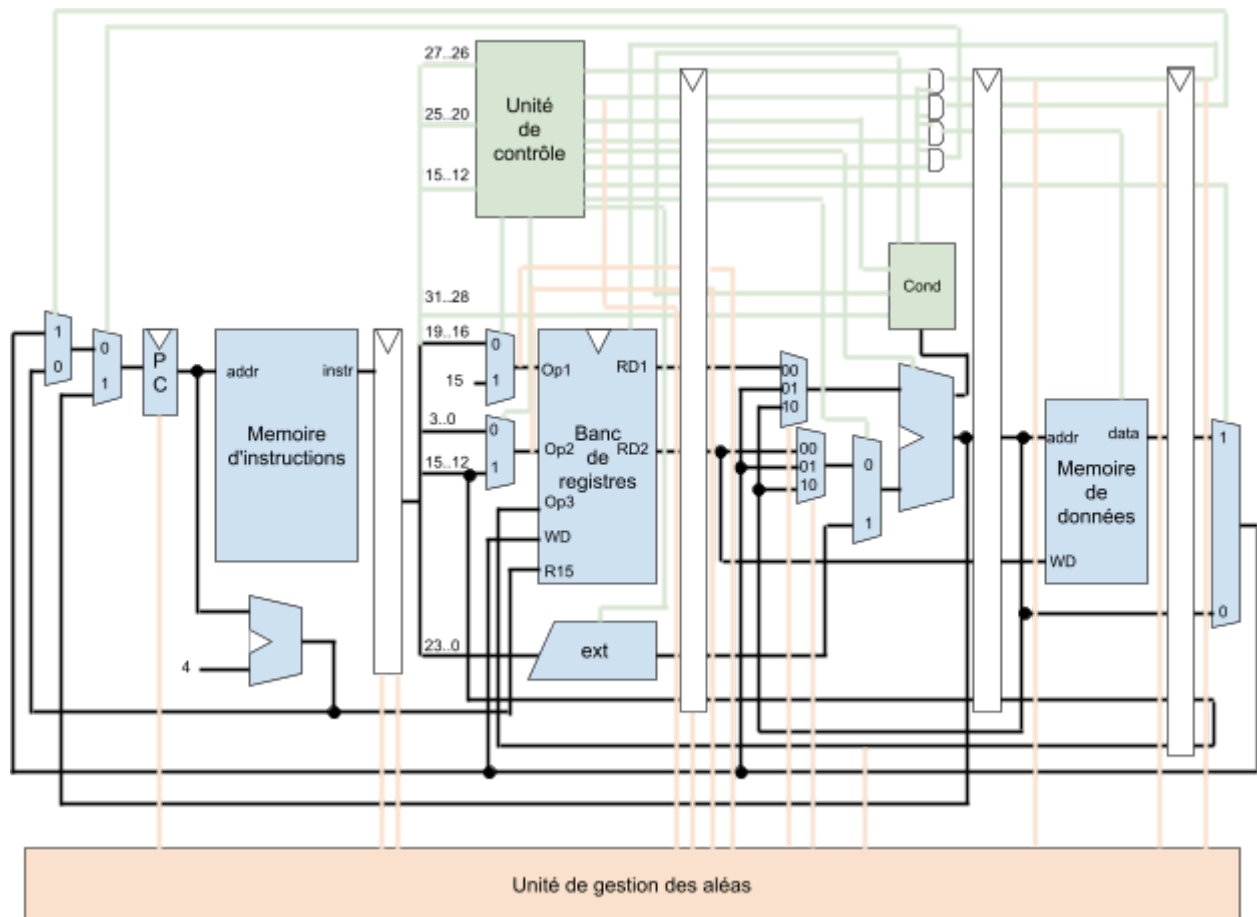


TP MACM – M1 SECIL

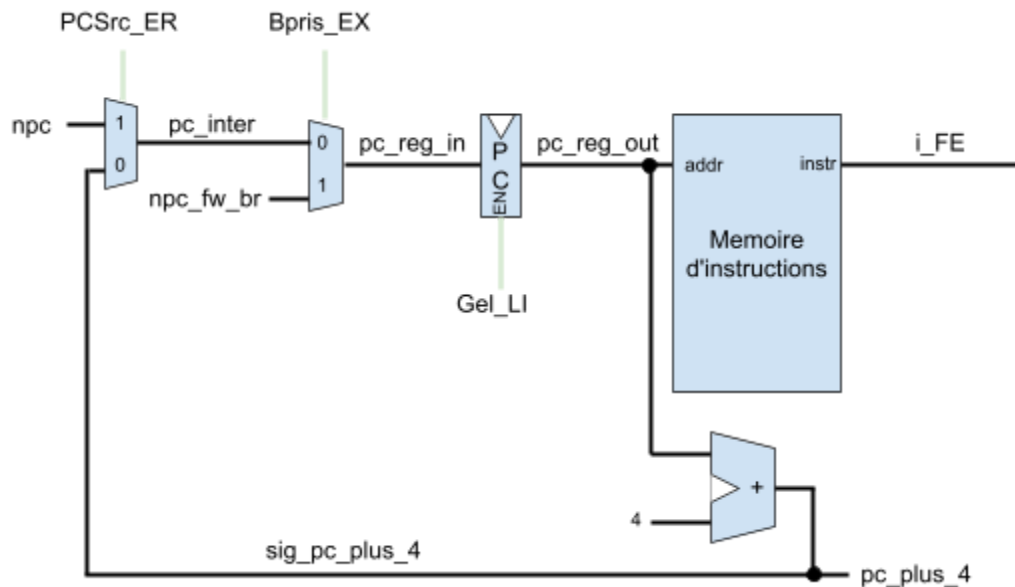
L'objectif de ces 5 séances de TP est de décrire en VHDL un processeur pipeliné dont le jeu d'instructions est basé sur l'ISA ARM (très simplifiée). On commencera par réaliser le chemin de données à partir de composants VHDL fournis, puis on développera la logique de contrôle. Enfin on rajoutera la logique de gestion des aléas de données.



I. Première partie : chemin de données

Dans cette première partie, nous allons instancier les principaux composants du chemin de données, fournis dans des fichiers .vhd.

I.1 Étage Fetch/Lecture d'Instruction



On va commencer par décrire l'étage de Fetch du processeur. Cela peut être fait dans un couple entité/architecture dédié, ou directement dans l'architecture du processeur. Dans ce TP, nous allons utiliser un couple entité/architecture dédié à chaque étage, ce qui facilitera les tests. Pour faciliter le travail, ce couple est déjà déclaré dans le fichier etages.vhd. Les noms des ports d'entrée/sortie du composant et des signaux intermédiaires figurant sur le schéma ci-dessus doivent être respectés.

Le PC courant est stocké dans un registre. Il permet de demander à la mémoire d'instructions de retourner l'instruction courante, qui sera passée à l'étage de décodage au cycle suivant. En parallèle, le prochain PC (sauf branchement ou instruction sur le pc) est calculé localement en ajoutant 4 au PC courant. Les autres valeurs possibles pour le prochain pc (en cas de branchement ou d'instruction écrivant dans le pc – e.g. add pc, pc, #16) sont calculées dans d'autres étages et apparaissent donc ici comme des ports d'entrée.

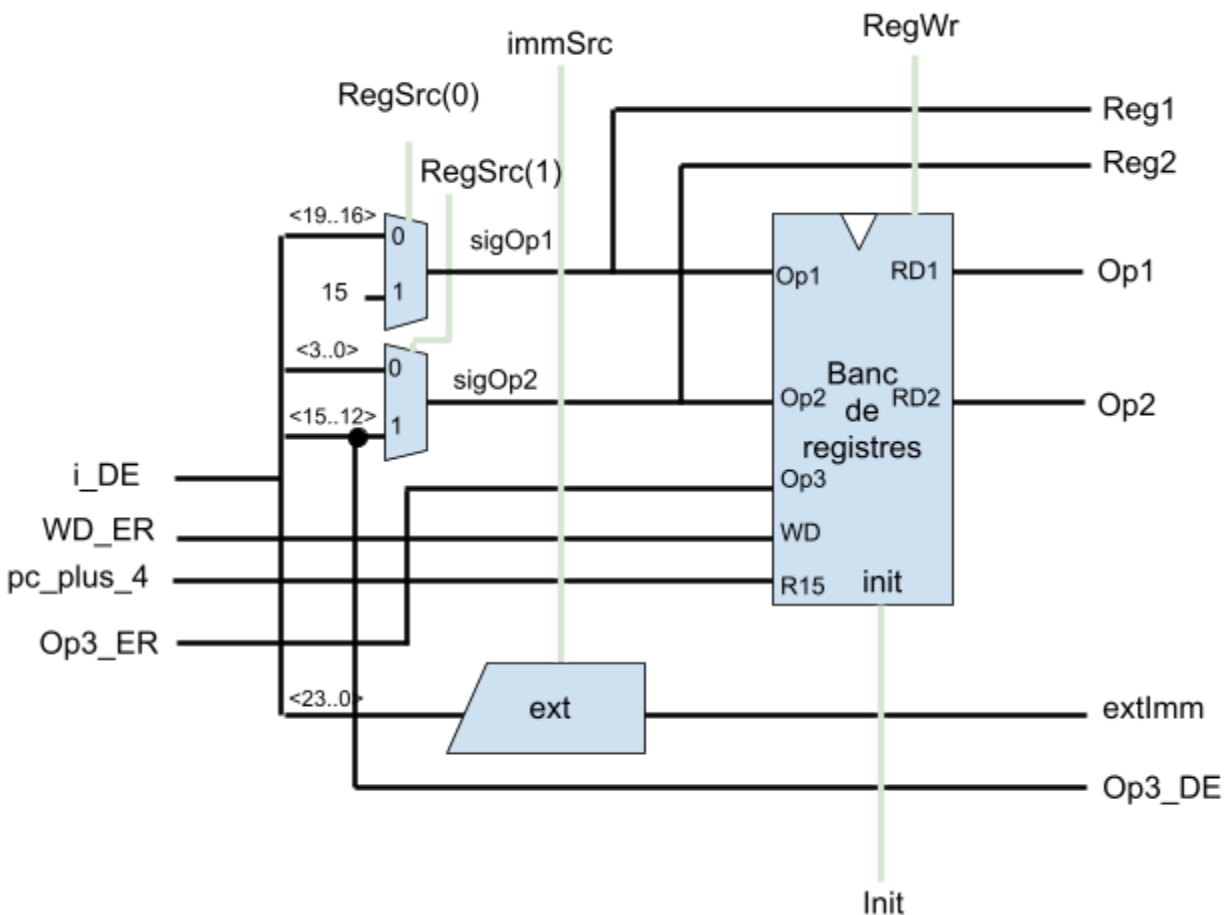
Résumé des ports d'entrée/sortie:

Port	E/S	Taille	Sémantique
npc	E	32 bits	Nouvelle valeur du pc calculée par une traversée complète du pipeline suite à une instruction arithmétique sur pc/r15 (ex: add pc, pc, #16)
npc_fw_br	E	32 bits	Nouvelle valeur du pc forwardée depuis l'étage EX, suite à un branchement
PC_Src_ER	E	1 bit	Signal de sélection du nouveau pc (en cas d'instruction sur le pc)
Bpris_EX	E	1 bit	Signal de sélection du nouveau pc (en cas de branchement)
GEL_LI	E	1 bit	Gel du registre pc en cas d'aléa dû à une lecture de donnée (0->gel, 1-> fonctionnement nominal)
clk	E	1 bit	Horloge du processeur
pc_plus_4	S	32 bits	Nouveau PC en cas d'exécution nominale
i_FE	S	32 bits	Instruction à décoder au cycle suivant

- Décrire l'étage de fetch dans le fichier etages.vhd (on utilisera en particulier l'entité inst_mem du fichier mem.vhd pour la mémoire d'instruction, et l'entité addComplex du fichier combi.vhd pour l'additionneur)

- Ecrire un banc de test pour tester le bon fonctionnement de l'étage de fetch. Vous pouvez modifier le fichier memfile.dat pour y inclure de vraies instructions ARM.

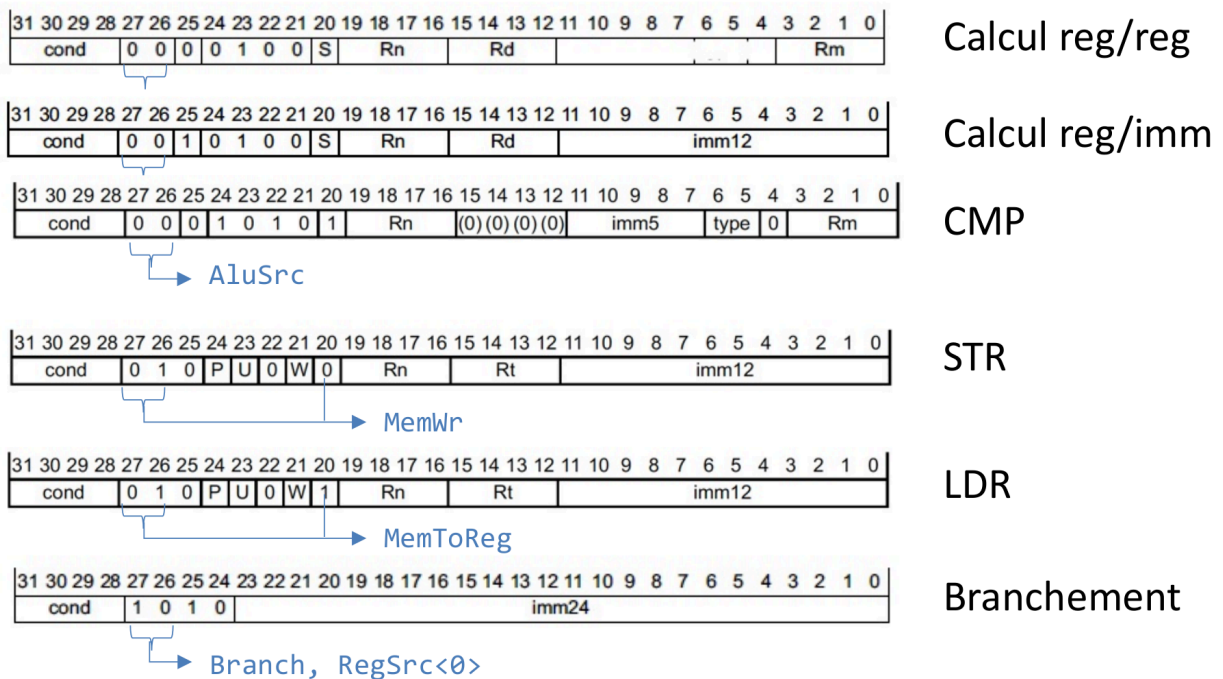
I.2 Étage Decode/Décodage des instructions



On va maintenant décrire l'étage de décodage. A partir d'une instruction *i_DE* passée par l'étage précédent, la logique de décodage extrait les champs permettant d'obtenir les opérandes de l'instruction (registres ou immédiat). De plus, le signal correspondant au registre destination est envoyé dans l'étage suivant (il va traverser le pipeline en même temps que l'instruction, et pourra configurer le banc de registre lors de l'écriture du résultat, via l'entrée *Op3_ER*). L'indice du registre de l'opérande 1 est soit fournie par l'instruction, soit le registre 15 (*pc*) lors d'un branchement (calcul de la cible du branchement relativement au *pc*). Ce choix est opéré par le signal de contrôle *RegSrc(0)*. L'indice du registre de l'opérande 2 peut être obtenue à deux endroits dans l'instruction, selon que celle-ci est une instruction de traitement de données ou une instruction mémoire. C'est le signal *RegSrc(1)* qui contrôle cette source. Les

données à écrire dans le banc de registre (WD_ER) proviennent du dernier étage du pipeline (valeur calculée ou donnée lue en mémoire). Enfin, la valeur du pc est transmise sans retard synchrone (i.e. dans le cycle d'horloge courant) par l'étage précédent.

Résumé du codage des instructions ARM (ISA simplifiée) :



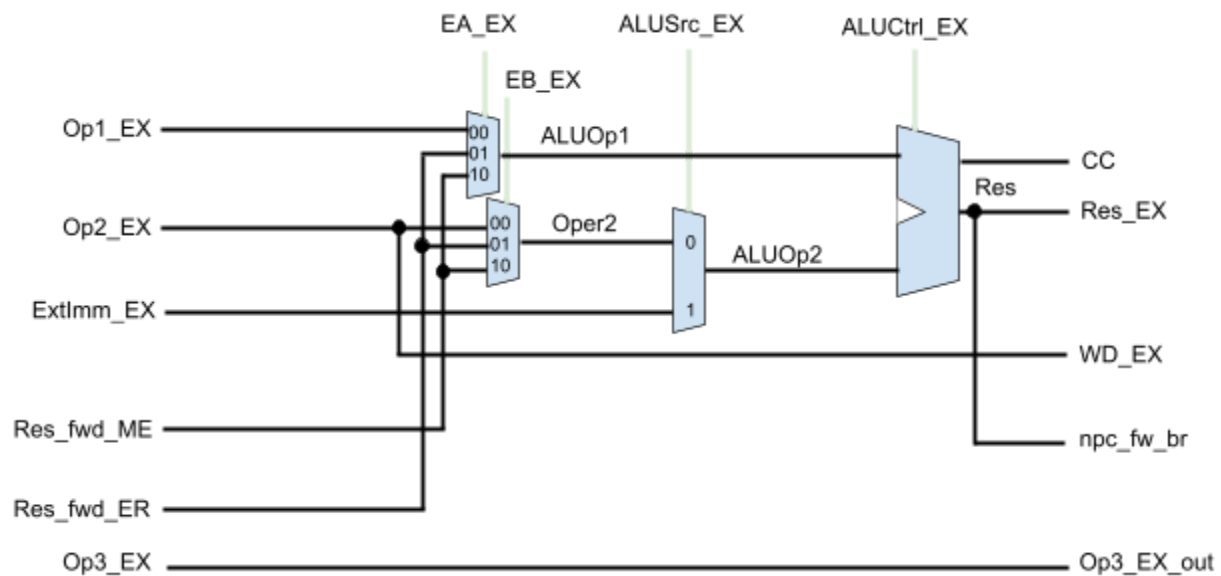
Résumé des ports d'entrée/sortie:

Port	E/S	Taille	Sémantique
i_DE	E	32 bits	Instruction à décoder
WD_ER	E	32 bits	Données à écrire
pc_plus_4	E	32 bits	Valeur du pc
Op3_ER	E	4 bits	Indice du registre à écrire à ce cycle
RegSrc	E	2 bits	Contrôle de la source des opérandes

immSrc	E	2 bits	Contrôle du mode d'extension des immédiats (signe + taille)
RegWr	E	1 bit	Contrôle d'écriture du banc de registre
clk	E	1 bit	Horloge du processeur
Init	E	1 bit	Initialisation asynchrone des registres
Reg1	S	4 bits	Indice de l'opérande 1 pour contrôle des aléas
Reg2	S	4 bits	Indice de l'opérande 2 pour contrôle des aléas
Op1	S	32 bits	Valeur de l'opérande 1
Op2	S	32 bits	Valeur de l'opérande 2
extImm	S	32 bits	Valeur étendue de l'immédiat
Op3_DE	S	4 bits	Indice du registre destination de l'instruction (à écrire dans 4 cycles)

- Décrire l'étage de décodage dans le fichier etages.vhd
- Ecrire un banc de test pour tester le bon fonctionnement de l'étage de décodage

I.3. Étage Execute



On s'attaque maintenant à l'étage d'exécution. Dans cet étage, l'objectif est de sélectionner les bonnes opérandes à envoyer dans l'ALU. La seconde opérande doit être sélectionnée entre un registre et un immédiat. Ce choix est réalisé par un multiplexeur en fonction du signal de contrôle ALUSrc_EX. Pour les opérandes registres, il faut choisir entre la valeur des registres venant de l'étage DE et les valeurs forwardées par l'étage Mémoire et l'étage Retire (qui peuvent contenir les valeurs nouvellement calculées mais non encore écrites des registres souhaités). Ces choix sont réalisés par deux multiplexeurs configurés par des signaux de contrôle (EA_EX et EB_EX) émis par l'unité d'envoi (qui gère les aléas).

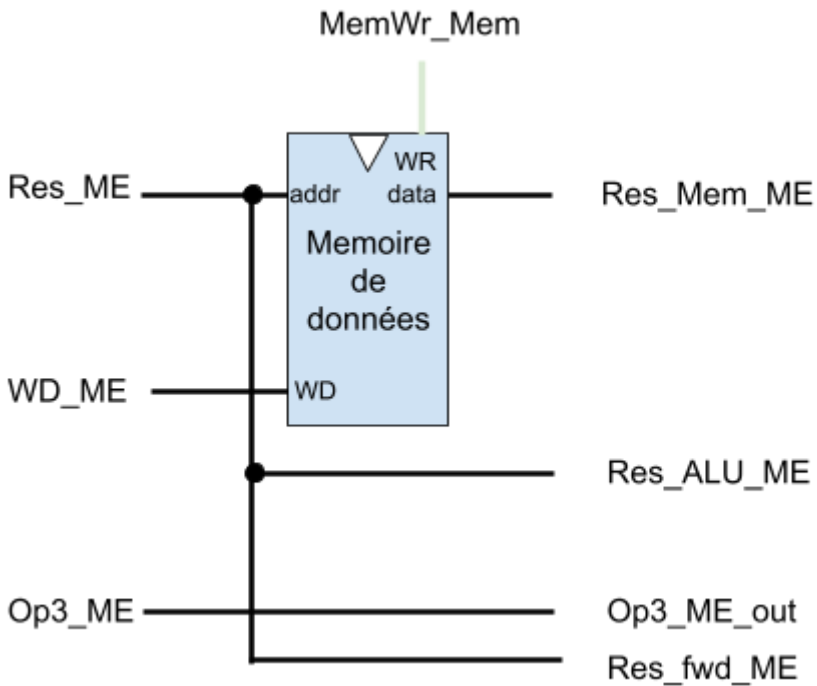
Résumé des ports d'entrée/sortie:

Port	E/S	Taille	Sémantique
Op1_EX	E	32 bits	Valeur de l'opérande 1 fournie par l'étage DE
Op2_EX	E	32 bits	Valeur de l'opérande 2 fournie par l'étage DE
ExtImm_EX	E	32 bits	Valeur immédiate fournie par l'étage DE
Res_fwd_ME	E	32 bits	Valeur du résultat calculé par l'étage EX

			au cycle précédent (forwardé depuis ME)
Res_fwd_ER	E	32 bits	Valeur du résultat en sortie de l'étage RE (forwardé depuis RE)
Op3_EX	E	4 bits	Indice du registre d'écriture de l'instruction en cours d'exécution (pour pouvoir écrire au bon endroit quand la valeur sera calculée)
EA_EX	E	2 bits	Sélection de l'opérande 1 avec résolution des aléas par forwarding
EB_EX	E	2 bits	Sélection de l'opérande 2 avec résolution des aléas par forwarding
ALUSrc_EX	E	1 bit	Sélection de l'opérande 2 entre registre et immédiat
ALUCtrl_EX	E	2 bits	Sélection de l'opération à réaliser dans l'ALU
CC	S	4 bits	Bits N, Z, C, V
Res_EX	S	32 bits	Sortie de l'ALU
WD_EX	S	32 bits	Valeur à écrire en mémoire
npc_fw_br	S	32 bits	Valeur calculée du nouveau pc en cas de branchement, à forwarder à l'étage FE
Op3_EX_out	S	4 bits	Indice du registre destination de l'instruction en cours d'exécution.

- Décrire l'étage d'exécution dans le fichier etages.vhd
- Ecrire un banc de test pour tester le bon fonctionnement de l'étage d'exécution

I.4 Étage Mémoire



La mémoire de données est un peu plus complexe que la mémoire d'instructions, car on doit pouvoir y réaliser des lectures et des écritures. Le signal **Res_ME** contient le résultat du calcul réalisé par l'ALU à l'étage précédent. Selon l'instruction, ce résultat est soit l'adresse à laquelle accéder (en lecture ou écriture), soit le résultat d'une instruction de traitement de données, auquel cas il doit être transmis à l'étage Retire, et forwardé à l'étage d'exécution pour résoudre de potentiels aléas. L'accès à la mémoire de données étant lent, il prend la majeure partie du temps de cycle. Si l'instruction en cours est une lecture mémoire, il n'est pas possible de forwarder la donnée lue vers l'étage d'exécution sans allonger le temps de cycle et donc sans ralentir le processeur. En cas de lecture, la donnée lue est forwardée au cycle suivant, depuis l'étage RE.

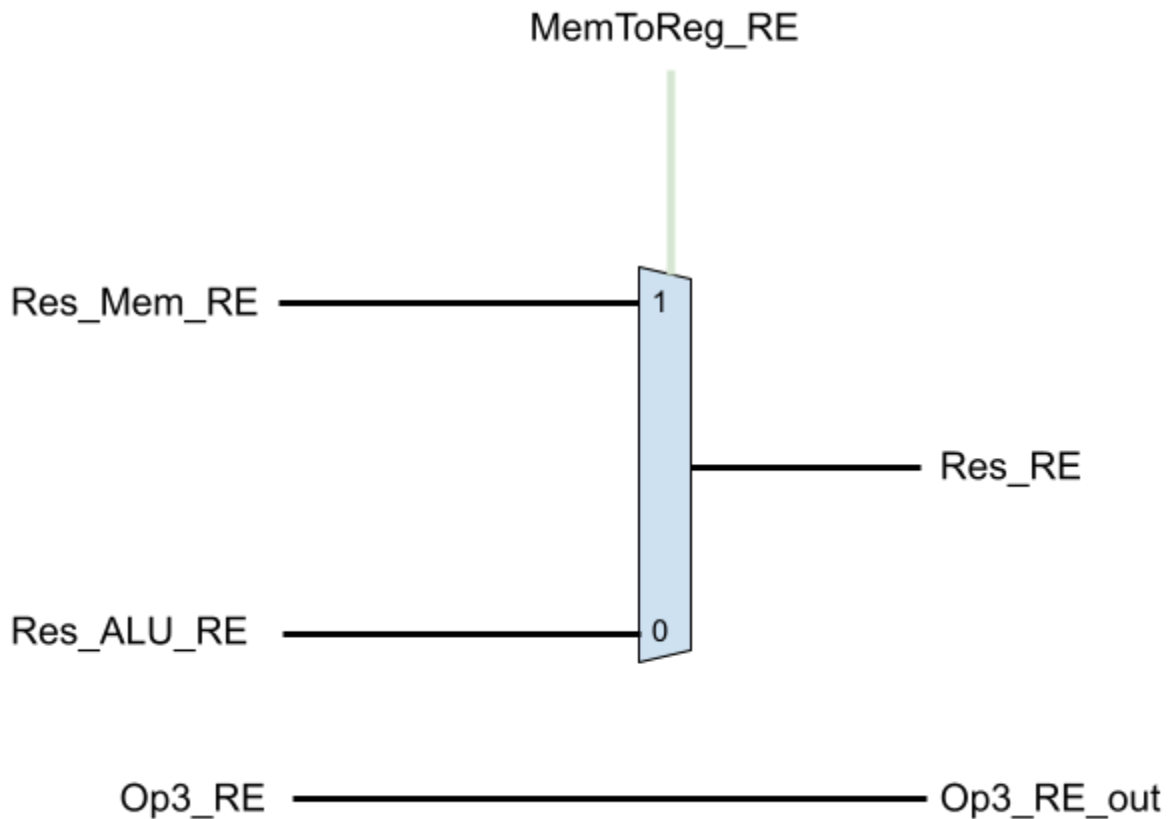
Résumé des ports d'entrée/sortie:

Port	E/S	Taille	Sémantique
Res_ME	E	32 bits	Résultat de l'ALU, passé par un registre. Il peut correspondre

			à l'adresse d'un accès mémoire, ou au résultat d'une instruction de traitement de données
WD_ME	E	32 bits	En cas de STR, c'est la donnée à écrire en mémoire
Op3_ME	E	4 bits	Indice du registre destination de l'instruction couramment dans l'étage ME (à transmettre pour pouvoir écrire le résultat dans le banc de registre)
clk	E	1 bit	Horloge du processeur
MemWR_Mem	E	1 bit	Signal d'écriture dans la mémoire (1-> str, 0-> toute autre instruction)
Res_Mem_ME	S	32 bits	Valeur lue en mémoire en cas de LDR
Res_ALU_ME	S	32 bits	Résultat de l'ALU au cycle précédent (au cas où l'instruction courante n'est pas un accès mémoire)
Op3_ME_out	S	4 bits	Indice du registre destination de l'instruction couramment dans ME
Res_fwd_ME	S	32 bits	Résultat de l'ALU au cycle précédent pour forward vers étage EX

- Décrire l'étage mémoire dans le fichier etages.vhd (on utilisera l'entité data_mem pour réaliser la mémoire de données)
- Ecrire un banc de test pour tester le bon fonctionnement de l'étage mémoire

I.5 Étage Retire



L'étage de retrait est très simple arrivée au bout du pipeline, l'instruction va en sortir. On sélectionne la valeur produite par l'instruction (entre instruction de traitement de données ou lecture mémoire) avec le signal de contrôle `MemToReg_RE`, et on transmet encore le signal `Op3` correspondant à l'indice du registre destination de l'instruction couramment dans l'étage RE.

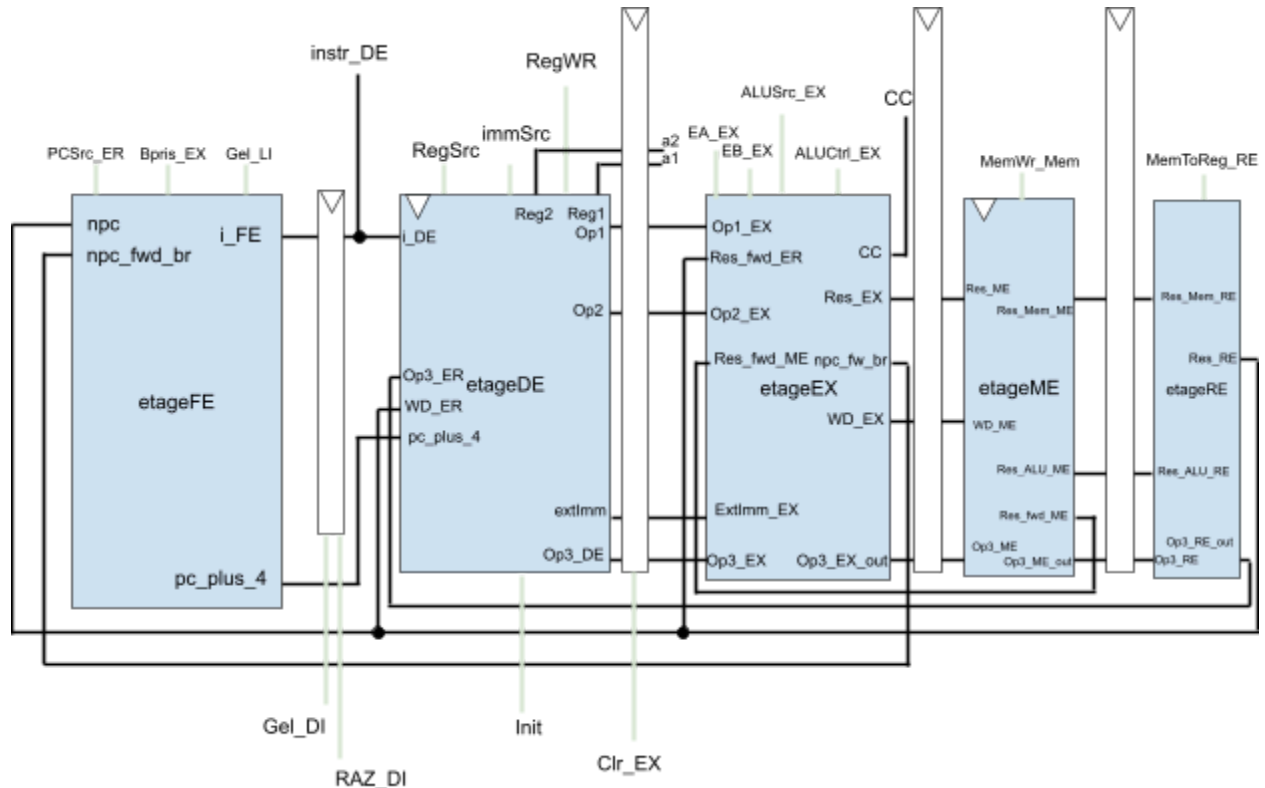
Résumé des ports d'entrée/sortie:

Port	E/S	Taille	Sémantique
<code>Res_Mem_RE</code>	E	32 bits	Résultat d'une lecture en mémoire
<code>Res_ALU_RE</code>	E	32 bits	Résultat du calcul de l'ALU pour

			l'instruction considérée
Op3_RE	E	4 bits	Indice du registre destination de l'instruction considérée
MemToReg_RE	E	1 bit	Signal de contrôle pour sélectionner le résultat
Res_RE	S	32 bits	Résultat de l'instruction considérée
Op3_RE_out	S	4 bits	Indice du registre destination, à transférer à l'étage DE

- Décrire l'étage RE dans le fichier etages.vhd
- Ecrire un banc de test pour tester le bon fonctionnement de l'étage RE

I.6 Chemin de données



On peut désormais réaliser le chemin de données en utilisant les étages réalisés précédemment, en insérant des registres pour retarder les signaux entre les étages lorsque c'est nécessaire, et en reliant les ports des étages par des signaux. Ce composant possède 17 ports d'entrée correspondant à l'horloge du processeur et à des signaux de contrôle que nous générerons dans un TP ultérieur, ainsi que 4 ports de sortie : l'instruction en cours de décodage (à destination de l'unité de contrôle), les indices des registres opérands de l'instruction en cours d'exécution (**a1** et **a2**), à destination de l'unité d'envoi (résolution des aléas) et les bits **CC** (**N**, **Z**, **C**, **V**) à destination de l'unité de calcul de condition.

- Réalisez le chemin de données dans le fichier **proc.vhd**, en vous servant des ports et des signaux déjà déclarés.
- Configurez la mémoire d'instructions en modifiant le fichier **memfile.dat** et testez votre chemin de données sur des exemples simples.